# WAS Experiment 02

**Aim:** Crawling and Scanning using Burp Suite

**Learning Objective**: To demonstrate how Burp Suite can be used to crawl a web application, identify its accessible resources, analyze the application's attack surface, and perform security scanning in a controlled environment.

**Learning Objectives:**
After completing this experiment, students will be able to:

- Understand the concept of web application crawling.
- Understand the purpose of scanning in web security testing.
- Identify URLs, directories, parameters, and resources of a web application.
- Use Burp Suite to crawl a web application.
- Analyse the discovered application structure.
- Perform security scanning using Burp Suite.
- Identify potential security issues in a web application.
- Understand the importance of authorized security testing.

**Tools:** Kali Linux, Burp Suite, Web Browser - Firefox, a local or authorized web application and Internet connection.

**Theory:**

**Introduction**

Web applications consist of multiple pages, directories, resources, parameters, forms, and endpoints. Before performing security testing, it is important to understand the structure and attack surface of the application.

Crawling is the process of systematically discovering the pages, links, resources, and endpoints associated with a web application.

Scanning is the process of analysing the discovered application components to identify potential security vulnerabilities or weaknesses.

Burp Suite provides tools that can assist security testers in performing both activities during an authorized web application security assessment.

**What is Crawling?**

Crawling is the process of discovering the structure and content of a web application by following links and observing requests made by the application.

During crawling, information such as the following may be discovered:

- Web pages
- URLs
- Directories
- Forms
- Parameters
- HTTP requests
- Application resources
- API endpoints

Crawling helps the tester understand the application's attack surface before conducting further security testing.

**What is Scanning?**

Scanning involves analysing the discovered web application components for potential security vulnerabilities.

A security scanner may examine areas such as:

- Input parameters
- HTTP requests and responses
- Application configuration
- Authentication mechanisms
- Session management
- Security headers
- Common web vulnerabilities

The results of scanning can then be reviewed by the security tester.

**Procedure:**

**Step 1: Start Kali Linux**

Start Kali Linux and open the desktop environment.

Launch Burp Suite from the application menu or using the terminal.

burpsuite

Expected Output:

Burp Suite starts successfully.

**Step 2: Create a Burp Suite Project**

Select:

Temporary Project

Use the default configuration and continue.

Expected Output:

The main Burp Suite interface is displayed.

**Step 3: Configure the Browser**

Configure Firefox to use Burp Suite as its local proxy.

Use:

Proxy Address: 127.0.0.1

Proxy Port: 8080

Expected Output:

Firefox traffic is routed through Burp Suite.

**Step 4: Open the Target Web Application**

Open the authorized target web application in Firefox.

For a local application, an example is:

http://localhost/

Only applications for which permission has been obtained should be tested.

Expected Output:

The target web application loads successfully.

**Step 5: Browse the Application**

Navigate through the available pages and links of the target application.

Open different sections, forms, and pages so that Burp Suite can observe the generated HTTP requests.

Expected Output:

HTTP requests generated by the browser are recorded by Burp Suite.

**Step 6: Open the Target/Site Map**

In Burp Suite, open the area containing the discovered application structure.

The discovered resources can be examined to understand the application's architecture.

Expected Output:

The application structure and discovered resources are displayed.

**Step 7: Perform Crawling**

Allow Burp Suite to discover additional accessible resources of the authorized target.

The crawler can identify resources such as:

- Pages
- Links
- Directories
- Parameters
- Requests

Expected Output:

Additional application resources are discovered and added to the application map.

**Step 8: Analyse the Discovered Resources**

Review the discovered URLs and requests.

Observe information such as:

- HTTP methods
- URL paths
- Parameters
- Request headers
- Response status codes

Expected Output:

The application's attack surface can be analysed from the discovered resources.

**Step 9: Select the Target for Scanning**

Select the authorized target or specific discovered resources that need to be scanned.

Start the appropriate Burp Suite scanning functionality available in the installed edition.

Expected Output:

Burp Suite begins analysing the selected target.

**Step 10: Analyse Scan Results**

After scanning is completed, review the reported findings.

Potential findings may be categorized according to their severity, such as:

- High
- Medium
- Low
- Informational

Expected Output:

Security findings are displayed for further analysis.

**Step 11: Examine an Identified Issue**

Select a reported finding and examine its details.

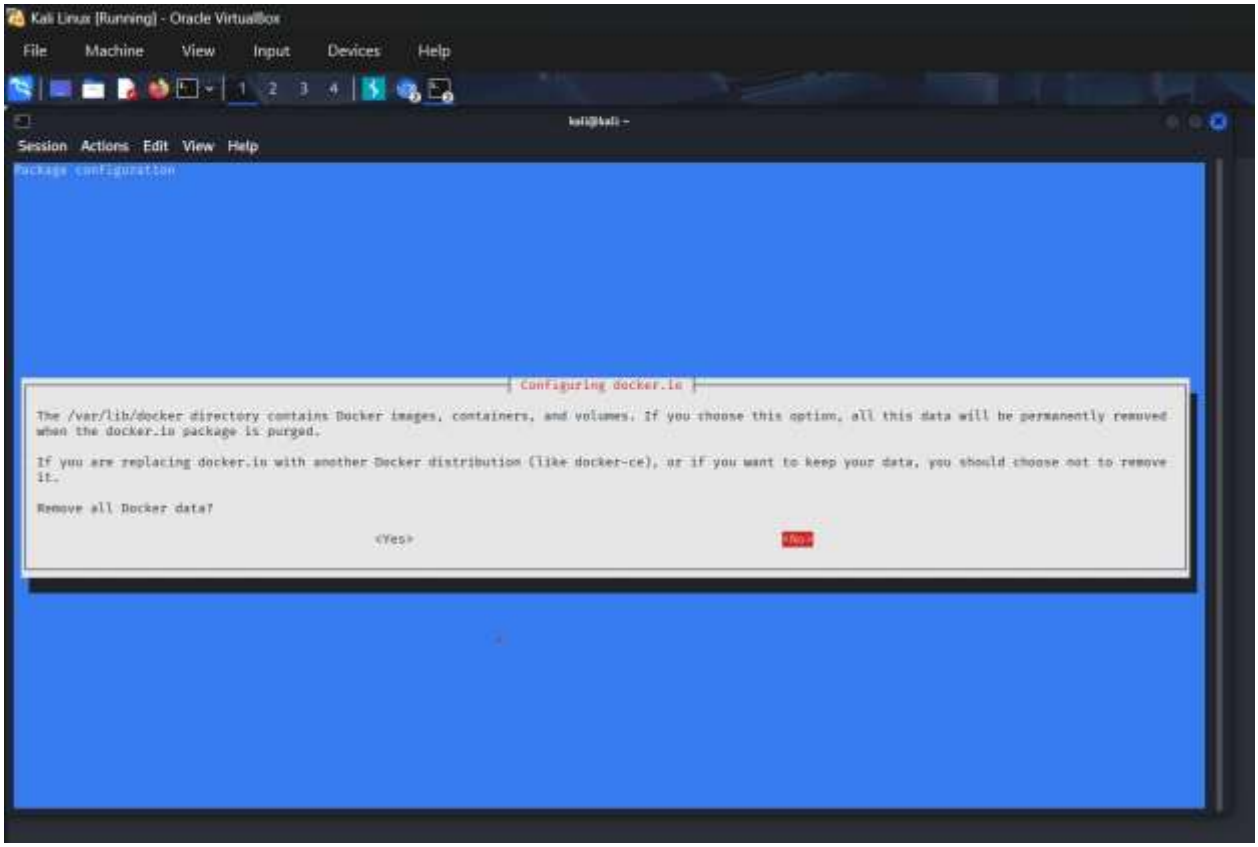
Review information such as:

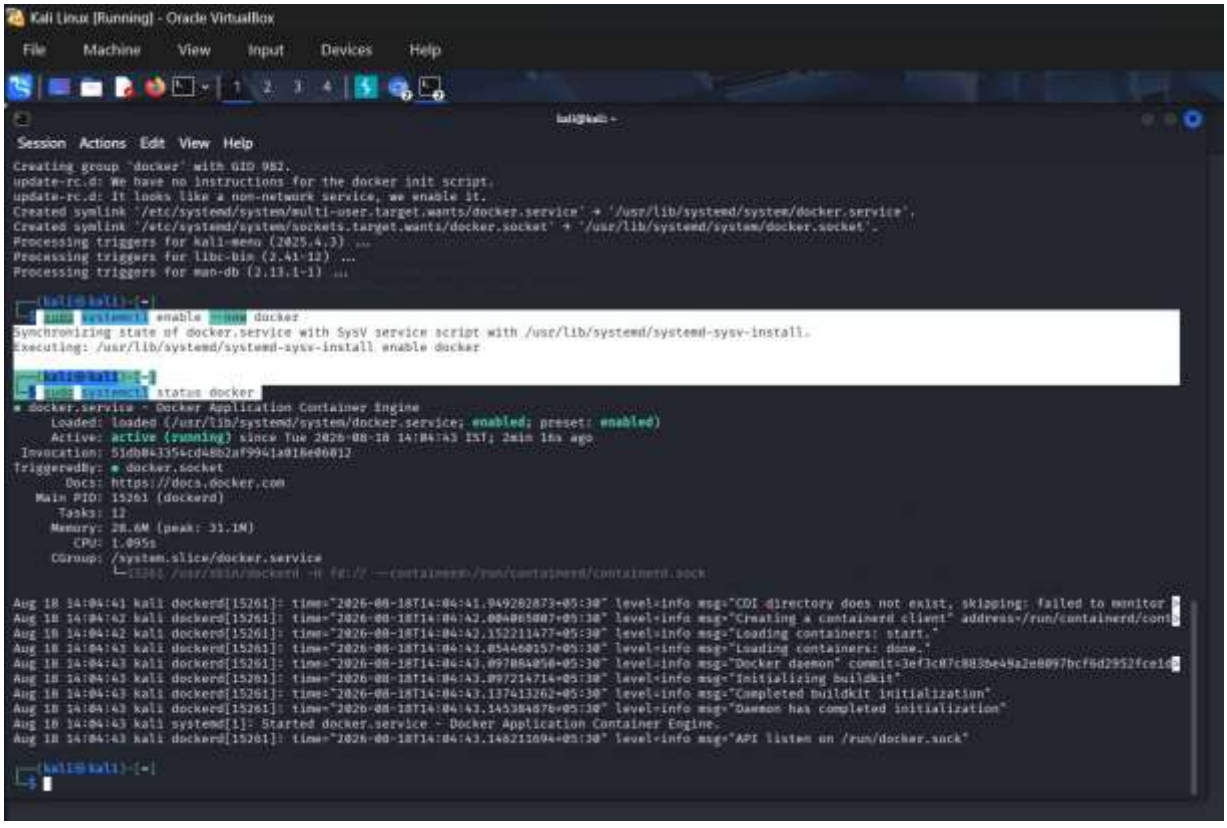
- Affected URL
- HTTP request
- HTTP response
- Issue description
- Severity
- Evidence
- Remediation information

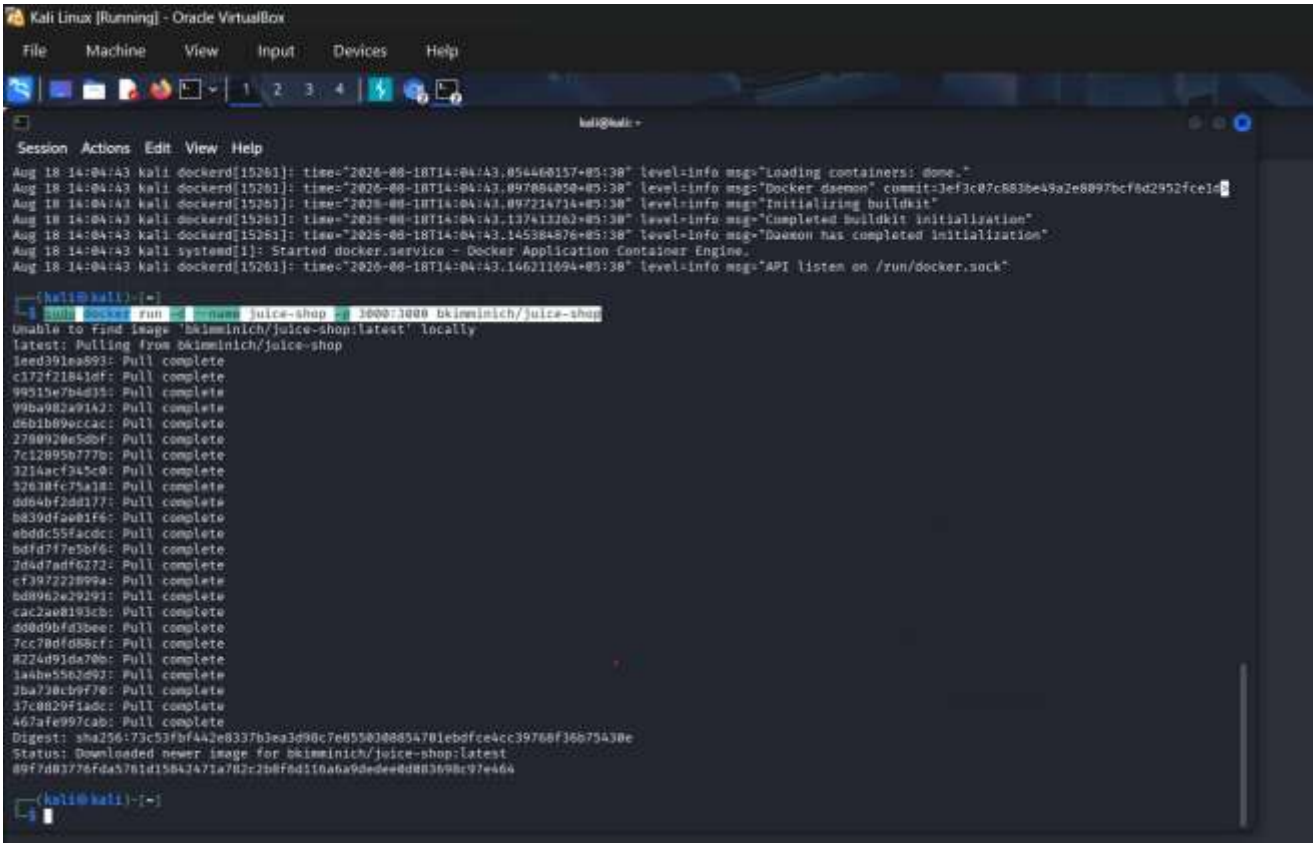
Expected Output:

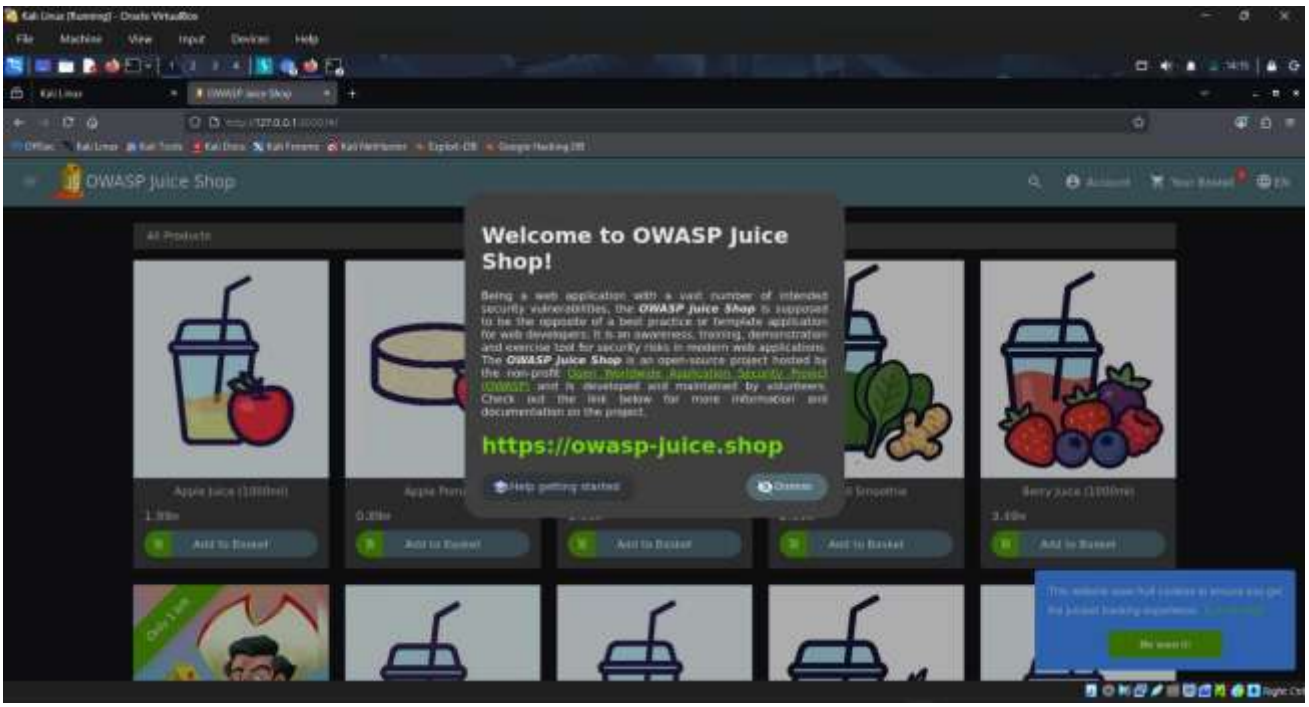
Detailed information about the selected security finding is displayed.

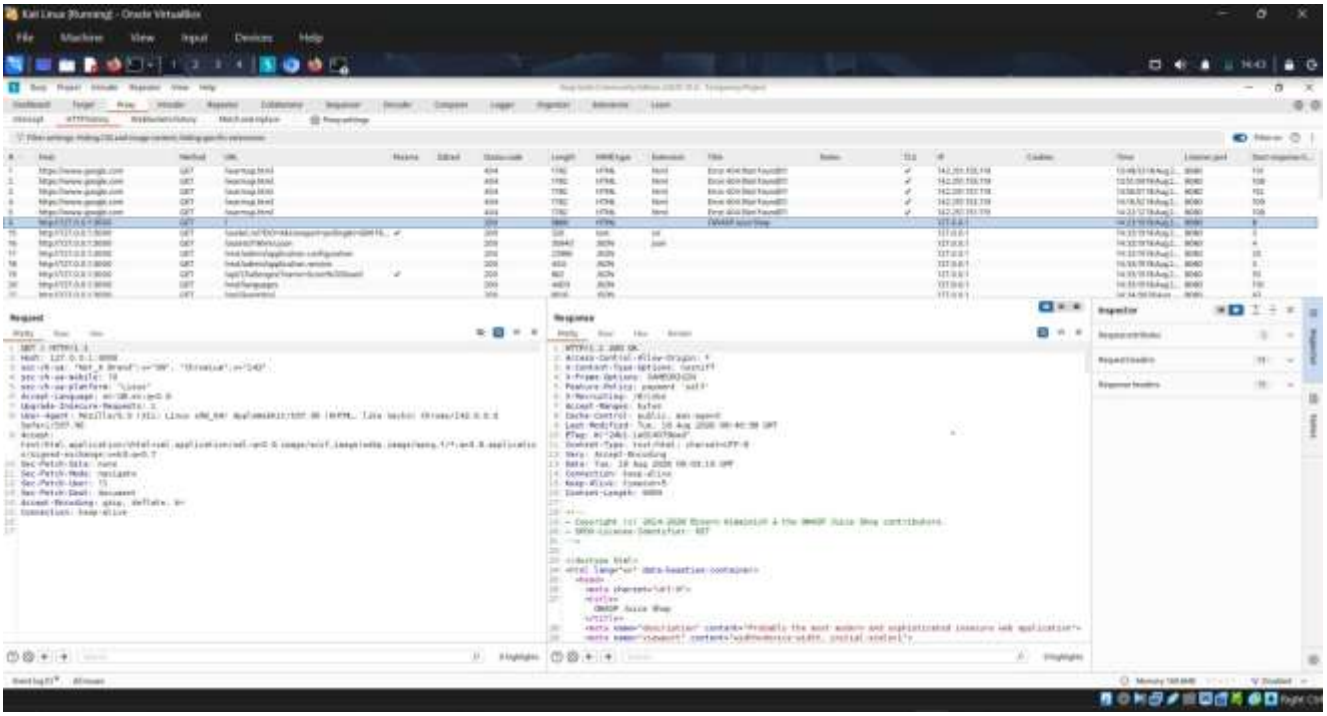
**Outputs:**

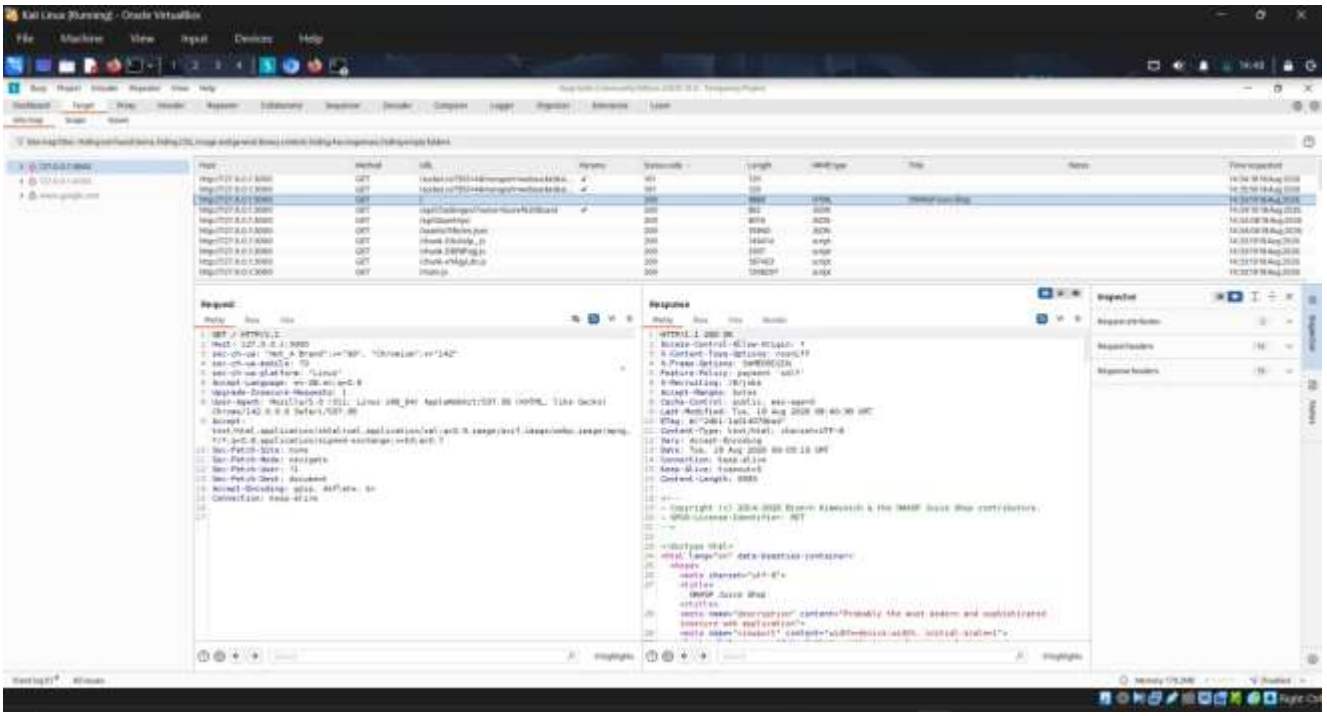

Kali Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Session Actions Edit View Help
Package configuration
Configuring docker.io
The /var/lib/docker directory contains Docker images, containers, and volumes. If you choose this option, all this data will be permanently removed when the docker.io package is purged.
If you are replacing docker.io with another Docker distribution (like docker-ce), or if you want to keep your data, you should choose not to remove it.
Remove all Docker data?
<Yes>
<No>


Kali Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Session Actions Edit View Help
Creating group 'docker' with GID 982.
update-rc.d: We have no instructions for the docker init script.
update-rc.d: It looks like a non-network service, we enable it.
Created symlink '/etc/systemd/system/multi-user.target.wants/docker.service' → '/usr/lib/systemd/system/docker.service'.
Created symlink '/etc/systemd/system/sockets.target.wants/docker.socket' → '/usr/lib/systemd/system/docker.socket'.
Processing triggers for kali-menu (2025.4.3) ...
Processing triggers for libc-bin (2.41-12) ...
Processing triggers for man-db (2.13.1-1) ...
sudo systemctl enable --now docker
Synchronizing state of docker.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable docker
sudo systemctl status docker
● docker.service - Docker Application Container Engine
Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
Active: active (running) since Tue 2026-08-18 14:04:43 IST; 2min 16s ago
Invocation: 51db043356cd48b2af9961a016e00012
TriggeredBy: ● docker.socket
Docs: https://docs.docker.com
Main PID: 15261 (dockerd)
Tasks: 12
Memory: 28.6M (peak: 31.1M)
CPU: 1.095s
CGroup: /system.slice/docker.service
└─15261 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock
Aug 18 14:04:41 kali dockerd[15261]: time="2026-08-18T14:04:41.949202873+05:30" level=info msg="CDI directory does not exist, skipping: failed to monitor
Aug 18 14:04:42 kali dockerd[15261]: time="2026-08-18T14:04:42.084065087+05:30" level=info msg="Creating a containerd client" address=/run/containerd/cont
Aug 18 14:04:42 kali dockerd[15261]: time="2026-08-18T14:04:42.152211477+05:30" level=info msg="Loading containers: start."
Aug 18 14:04:43 kali dockerd[15261]: time="2026-08-18T14:04:43.054460157+05:30" level=info msg="Loading containers: done."
Aug 18 14:04:43 kali dockerd[15261]: time="2026-08-18T14:04:43.097086050+05:30" level=info msg="Docker daemon" commit=3ef1c87c883be48a2e8097bcf6d2952fce1d
Aug 18 14:04:43 kali dockerd[15261]: time="2026-08-18T14:04:43.097214714+05:30" level=info msg="Initializing buildkit"
Aug 18 14:04:43 kali dockerd[15261]: time="2026-08-18T14:04:43.137413262+05:30" level=info msg="Completed buildkit initialization"
Aug 18 14:04:43 kali dockerd[15261]: time="2026-08-18T14:04:43.145384d7b+05:30" level=info msg="Daemon has completed initialization"
Aug 18 14:04:43 kali systemd[1]: Started docker.service - Docker Application Container Engine.
Aug 18 14:04:43 kali dockerd[15261]: time="2026-08-18T14:04:43.146211694+05:30" level=info msg="API listen on /run/docker.sock"


Kali Linux [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Session Actions Edit View Help
Aug 18 14:04:43 kali dockerd[15261]: time="2026-08-18T14:04:43.054460157+05:30" level=info msg="Loading containers: done."
Aug 18 14:04:43 kali dockerd[15261]: time="2026-08-18T14:04:43.097084050+05:30" level=info msg="Docker daemon" commit=3ef3c07c883be49a2e8897bcf8d2952fce1d
Aug 18 14:04:43 kali dockerd[15261]: time="2026-08-18T14:04:43.097214734+05:30" level=info msg="Initializing buildkit"
Aug 18 14:04:43 kali dockerd[15261]: time="2026-08-18T14:04:43.137413262+05:30" level=info msg="Completed buildkit initialization"
Aug 18 14:04:43 kali dockerd[15261]: time="2026-08-18T14:04:43.145384876+05:30" level=info msg="Daemon has completed initialization"
Aug 18 14:04:43 kali systemd[1]: Started docker.service - Docker Application Container Engine.
Aug 18 14:04:43 kali dockerd[15261]: time="2026-08-18T14:04:43.146211694+05:30" level=info msg="API listen on /run/docker.sock"
sudo docker run -d --name juice-shop -p 3000:3000 bkimminich/juice-shop
Unable to find image 'bkimminich/juice-shop:latest' locally
latest: Pulling from bkimminich/juice-shop
Digest: sha256:73c53fbf442e8337b3ea3d98c7e85503885470lebdfce4cc39768f36b75430e
Status: Downloaded newer image for bkimminich/juice-shop:latest


OWASP Juice Shop
Welcome to OWASP Juice Shop!
https://owasp-juice.shop





**Observation:**

The following observations are made:

1. Burp Suite successfully captured web application traffic.
2. The target application's pages and resources were discovered.
3. The discovered resources were displayed in the application structure.

4. HTTP requests and responses could be analysed.
5. The authorized application could be subjected to security scanning.
6. Potential security findings were displayed by Burp Suite.
7. The discovered information helped in understanding the application's attack surface.

**Result and Discussion:**

The experiment successfully demonstrated crawling and scanning using Burp Suite.
During crawling, the web application's accessible resources, URLs, pages, and requests were identified. The discovered information provided an overview of the application's attack surface.
The scanning process analysed the selected application components and generated security findings for further investigation. The results can help security testers identify potential weaknesses and determine areas that require additional testing.
Crawling and scanning are important stages of web application security testing because they help testers understand the application before performing detailed vulnerability assessment.

**Conclusion:**

This experiment demonstrated the use of Burp Suite for crawling and scanning a web application in a controlled environment. The experiment showed how application resources can be discovered, mapped, and analysed and how security scanning can be used to identify potential security issues.
The experiment also emphasized that crawling and scanning should be performed only against web applications for which proper authorization has been obtained.

**Viva Questions:**

1. What is crawling in web application security?
2. What is scanning in Burp Suite?
3. What is the difference between crawling and scanning?

**For Faculty Use:**

| Correction Parameters | Formative Assessment [40%] | Timely completion of Practical [40%] | Attendance / Learning Attitude [20%] | |
|---|---|---|---|---|
| Marks Obtained | | | | |